



ITI · AI AGENTS DEVELOPMENT USING PYTHON TRACK

AI-Assisted Development & API Integration

Eng. Fatma Elraey*Lecture 1 — The AI-Assisted Developer (Vibe Coding Workflow)*

Field	Detail
Instructor	Fatma Elraey
Course	AI-Assisted Development & API Integration (2-Day Course)
Session	Day 1 of 2 — Lecture, 3 Hours
Focus	Mastering the industry-standard AI coding tools (Cursor, Copilot) that make developers dramatically faster, with a clear-eyed view of their limits and where human review still matters.

Welcome!

This course assumes you can already write and read basic code. Today isn't about learning a new programming language — it's about learning a new way of working with one: writing software alongside an AI assistant instead of typing every line by hand. By the end of today you'll understand the vibe coding workflow, know your way around Cursor and GitHub Copilot's different modes, write prompts that get better code out of them, and know when a no-code tool is the right call instead.

Today's Outline

1. The Shift to AI-Assisted Development
2. The Cursor & Copilot Ecosystem
3. Prompting for Code
4. No-Code / Low-Code Rapid Prototyping

01

The Shift to AI-Assisted Development

"Vibe coding" is the informal name for a style of development where you describe what you want in plain English, let an AI assistant generate the actual code, and focus your own effort on steering, reviewing, and testing rather than typing every character. The term is casual, but the shift it describes is real: a huge share of professional developers now write code with an AI assistant active most of the time.

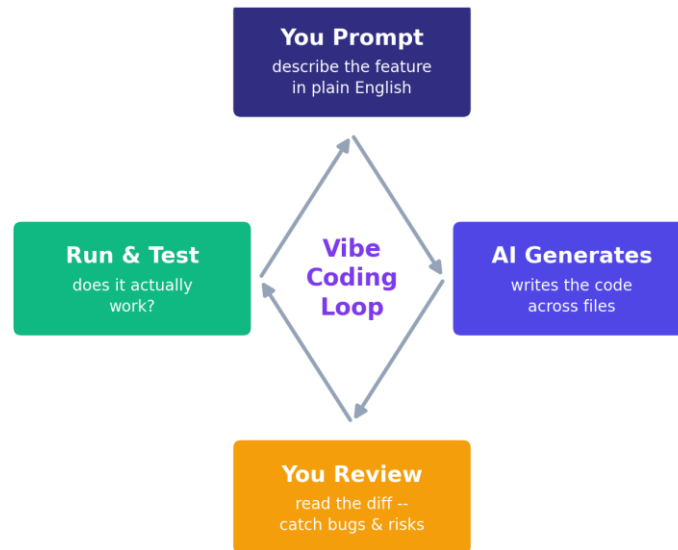


Figure 1.1 — The vibe coding loop: you steer and check, the AI writes.

The **benefits** are genuinely large when this workflow is used well:

- **Speed** -- boilerplate, repetitive patterns, and first drafts appear in seconds
- **Lower barrier** -- unfamiliar languages or frameworks become approachable
- **Focus** -- you spend more time on design and logic, less on syntax lookup

But the workflow has real limitations, and this is the part most beginners skip past too quickly:

- **Confident but wrong** -- AI-generated code can look correct and still contain subtle bugs, security holes, or outdated practices.
- **No real understanding** -- the AI doesn't know your business rules or the parts of your codebase it can't see.
- **Skill atrophy risk** -- blindly accepting suggestions without reading them slowly erodes your own ability to catch problems.



```
# Instructor examples – key terms
```

BOILERPLATE

```
if __name__ == "__main__":  
    main()
```

A repeated, standard pattern that usually needs little new logic.

SKILL ATROPHY

If AI writes your loops, functions, and debugging every time and you only press Accept, your own coding skill can weaken.

DIFF

Before:

```
def add(a, b):  
    return a + b
```

After:

```
def add(a, b):  
    if a is None or b is None:  
        return 0  
    return a + b
```

The diff is the set of lines added, removed, or changed.

This is exactly why **developer oversight** isn't optional — it's the actual skill this course is teaching. Two habits matter most: reading every non-trivial diff before accepting it (the same way you'd review a teammate's pull request), and a basic security pass — checking for hardcoded secrets, unsanitized user input, and overly broad permissions before anything ships.

```
# Instructor example – security review
```

Avoid hardcoded secrets:

```
API_KEY = "abc123..."
```

Avoid building SQL directly from user input:

```
query = f"SELECT * FROM users WHERE name = '{name}'"
```

Review secrets, input handling, and permissions before shipping.

PRO TIP *Treat AI-generated code exactly like a pull request from a junior developer: probably fine, occasionally brilliant, sometimes confidently wrong. Your job doesn't disappear — it moves from typing to reviewing.*



✓ QUICK CHECK — The Shift to AI-Assisted Development

Q1. What is 'vibe coding,' in plain terms?

Answer: Describing what you want in plain English and letting an AI assistant generate the code, while you steer, review, and test.

Q2. Why does developer oversight remain critical even with a good AI assistant?

Answer: AI-generated code can look correct while containing subtle bugs or security issues, and the AI doesn't know your specific business rules or unseen parts of your codebase.

02

The Cursor & Copilot Ecosystem

Cursor and GitHub Copilot are today's two industry-standard AI coding assistants — Cursor is a full code editor built around AI from the ground up, while Copilot is an extension that adds AI features to editors like VS Code. Both give you the same three core ways of working with AI, just under slightly different names.

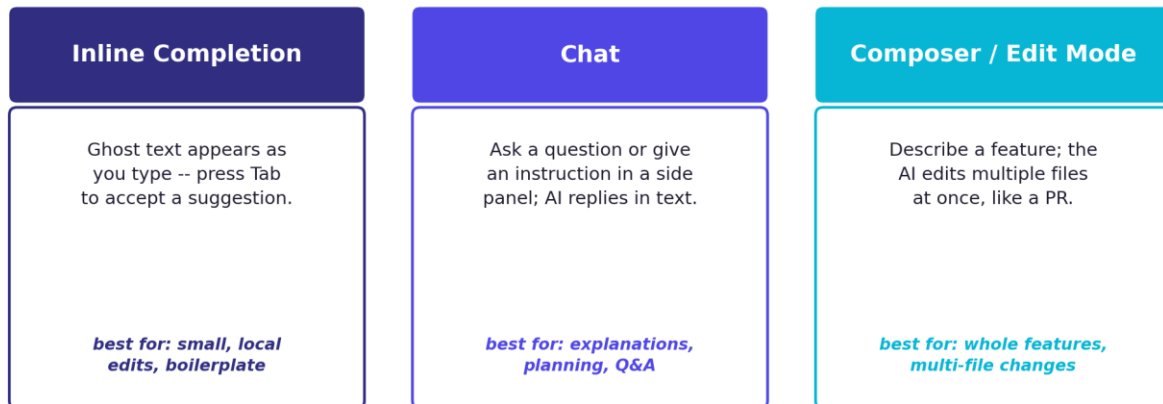


Figure 1.2 — Three modes, three different jobs.

Inline completion is the lightest-touch mode: as you type, the assistant suggests the rest of the line or the next few lines as grayed-out "ghost text." Press Tab to accept it, or just keep typing to ignore it. It shines for boilerplate — repetitive patterns the assistant has seen a thousand times before.

```
# Instructor example – inline completion
YOU TYPE
def calculate_total(items):

AI SUGGESTS AS GHOST TEXT
    return sum(item.price for item in items)
```

Press Tab if the suggestion is correct; otherwise keep typing.

Chat is a conversation, usually in a side panel. You ask questions, request explanations, or describe a small change, and the AI replies in text — sometimes with a code snippet you copy in yourself, sometimes with a proposed edit you can accept with one click.

```
# Instructor example – Chat
Why can this line raise KeyError?
data["username"]
```

Show me three safe alternatives and explain when to use each.

Composer (Cursor's name) or Edit Mode (Copilot's name for a similar feature) is the most powerful of the three: you describe an entire feature, and the AI proposes changes across multiple files at once — closer to reviewing a small pull request than accepting a single suggestion.

```
# Instructor example — Composer / Edit Mode
```

```
Add password-reset functionality.
```

```
Update these files:
```

- backend/routes/auth.py
- backend/services/email.py
- frontend/pages/ForgotPassword.jsx
- frontend/pages/ResetPassword.jsx
- tests/test_password_reset.py

```
This is a feature-sized request spanning multiple files.
```

PRO TIP Match the mode to the size of the task. A one-line fix doesn't need Composer, and a brand-new feature spanning five files is painful to build one inline suggestion at a time.

✓ QUICK CHECK — The Cursor & Copilot Ecosystem

Q1. What is inline completion best suited for?

Answer: Small, local edits and boilerplate — repetitive patterns the assistant can predict as you type.

Q2. What makes Composer / Edit Mode different from Chat?

Answer: It can propose and apply changes across multiple files at once, closer to a small pull request, rather than a single text reply.



03

Prompting for Code

The same clarity principles from prompting a chatbot apply here, with one addition specific to code: the AI can only work with the context it can actually see. A vague request forces it to guess your conventions, your file structure, and your intent.

A weak prompt for a coding assistant:

```
Add login to my app.
```

A strong prompt gives system context — the framework, the constraints, and the expected shape of the result:

```
Add a login form to the React component in
src/pages/Login.jsx. Use the existing useAuth()
hook from src/hooks/useAuth.js for the actual
authentication call. Match the styling already
used in src/pages/Signup.jsx. Show a loading
spinner while the request is in flight.
```

```
# Instructor example – another strong coding prompt
In api/users.py, GET /users/{id} currently returns 500
when the user does not exist.
```

Change it to return HTTP 404.

Keep the existing response schema unchanged.
Add a unit test for the missing-user case.

Notice: file + current bug + expected result + constraint + test.

For anything beyond a one-off request, a rules file is far more efficient than repeating context in every prompt. Both tools support a project-level file (Cursor uses `.cursorrules`, Copilot uses `.github/copilot-instructions.md`) that's automatically included as context for every request in that project:

```
# .copilot-instructions.md
This project uses React 18 with TypeScript and
Tailwind CSS. State management is React Query --
do not introduce Redux. All API calls go through
the client in src/lib/apiClient.ts. Prefer
functional components with hooks; no class
components. Write a unit test for every new
utility function.
```

For multi-file edits, be explicit about which files are in scope. Assistants can only edit what they can see, so pointing directly at relevant files ("update the API route in `api/users.ts` and the matching types in `types/user.ts`") produces far more consistent results than a vague "update the backend."



PRO TIP *A rules file is a one-time, five-minute investment that pays off on every single prompt you write in that project afterward. Write one for any project you'll touch more than a couple of times.*

✓ QUICK CHECK — Prompting for Code

Q1. What's the main advantage of a rules file (.cursorrules or copilot-instructions.md) over repeating context in every prompt?

Answer: *It's included automatically as context for every request in the project, so you don't have to restate your framework, conventions, and constraints each time.*

Q2. Why should you name specific files when asking for a multi-file change?

Answer: *Assistants can only edit what they can see — naming files explicitly produces far more consistent results than a vague request to 'update the backend.'*

04

No-Code / Low-Code Rapid Prototyping

A different category of tool skips writing code almost entirely: describe an application in plain English, and the tool generates a complete, working frontend — often deployed and shareable within minutes. These aren't replacements for a real engineering workflow, but they're excellent for one specific job: turning an idea into something clickable, fast.

```
# Instructor example – no-code prototype prompt
```

```
Build a dashboard with:
```

- a sidebar
- a sales chart
- a recent-orders table
- dark mode

```
Use this to test the idea and UI quickly before production work.
```



Describe an app in plain English -- get a working frontend in minutes

Figure 1.3 — Describe it, get a working prototype.

The typical use case is proving out an idea before committing real engineering time: a founder validating a concept with early users, a developer mocking up a UI direction for feedback before building it properly, or a quick internal tool that doesn't need production-grade engineering.

The honest limitation: these tools optimize for speed to a working demo, not for the same **maintainability**, **security**, or **scalability** you'd want in production code. Treat what comes out as a prototype to learn from, not infrastructure to build a company on without further engineering.

PRO TIP *A good litmus test: if you'd be comfortable if this prototype got real user traffic and real user data tomorrow, you're probably past the point where a no-code tool alone is enough.*



✓ QUICK CHECK — No-Code / Low-Code Rapid Prototyping

Q1. What is the best use case for a tool like Bolt.new or Lovable?

***Answer:** Quickly turning an idea into a clickable prototype — validating a concept or getting UI feedback fast, not building production infrastructure.*

Q2. What's the main trade-off these tools make?

***Answer:** They optimize for speed to a working demo, generally at the cost of the maintainability, security, and scalability you'd want in production code.*



Check Your Understanding

Work through these on your own before checking the marked answer.

Practice Question 1

What best describes the 'vibe coding' workflow?

- A. Writing every line of code manually without any AI assistance
- B. Describing what you want in plain English and letting an AI assistant generate code while you steer and review
- C. Only using no-code tools and never writing code
- D. Letting an AI assistant deploy code directly to production with no human review

Practice Question 2

You need to add a small, one-line bug fix. Which AI-assistant mode is the best fit?

- A. Composer / Edit Mode, to be thorough
- B. Inline completion or a quick Chat question
- C. A no-code prototyping tool
- D. None of these modes can handle a bug fix

Practice Question 3

What is the main purpose of a rules file like `.cursorrules` or `copilot-instructions.md`?

- A. It replaces the need for developer code review entirely
- B. It automatically supplies project context (frameworks, conventions, constraints) to every AI request in that project
- C. It only works with no-code tools
- D. It's required before an AI assistant can be installed



Practice Question 4

A founder wants to quickly show early users a clickable idea before writing real production code. What's the best-fit tool category?

- A. A no-code / low-code prototyping tool (e.g. Bolt.new, Lovable)
- B. A production-grade backend framework
- C. Inline completion only
- D. None — this always requires a full engineering team first

Practice Question 5

Why is 'the AI-generated code looked correct, so I accepted it without reading it' a risky habit?

- A. It isn't risky — AI-generated code is always correct
- B. AI-generated code can be confidently wrong, containing subtle bugs or security issues that look fine at a glance
- C. It's only risky for no-code tools, not for Cursor or Copilot
- D. It's risky only when writing tests

Answers

Practice Question 1

What best describes the 'vibe coding' workflow?

- A. Writing every line of code manually without any AI assistance
- B. Describing what you want in plain English and letting an AI assistant generate code while you steer and review ✓**
- C. Only using no-code tools and never writing code
- D. Letting an AI assistant deploy code directly to production with no human review

Why: Vibe coding centers on natural-language steering plus AI-generated code, with the human's role shifting toward reviewing and testing rather than typing every character.



Practice Question 2

You need to add a small, one-line bug fix. Which AI-assistant mode is the best fit?

- A. Composer / Edit Mode, to be thorough
- B. Inline completion or a quick Chat question ✓**
- C. A no-code prototyping tool
- D. None of these modes can handle a bug fix

Why: A small, local, one-line change doesn't need the overhead of a multi-file Composer session — inline completion or a quick chat message is the right-sized tool.

Practice Question 3

What is the main purpose of a rules file like .cursorrules or copilot-instructions.md?

- A. It replaces the need for developer code review entirely
- B. It automatically supplies project context (frameworks, conventions, constraints) to every AI request in that project ✓**
- C. It only works with no-code tools
- D. It's required before an AI assistant can be installed

Why: A rules file is included automatically as context on every request, so you don't have to restate your project's conventions and constraints in each individual prompt.

Practice Question 4

A founder wants to quickly show early users a clickable idea before writing real production code. What's the best-fit tool category?

- A. A no-code / low-code prototyping tool (e.g. Bolt.new, Lovable) ✓**
- B. A production-grade backend framework
- C. Inline completion only
- D. None — this always requires a full engineering team first

Why: No-code/low-code tools are specifically optimized for turning an idea into a working, clickable demo fast — exactly this use case.



Practice Question 5

Why is 'the AI-generated code looked correct, so I accepted it without reading it' a risky habit?

- A. It isn't risky — AI-generated code is always correct
- B. AI-generated code can be confidently wrong, containing subtle bugs or security issues that look fine at a glance ✓**
- C. It's only risky for no-code tools, not for Cursor or Copilot
- D. It's risky only when writing tests

***Why:** One of the clearest limitations covered today is that AI-generated code can look plausible and well-formatted while still containing real bugs or security problems — which is exactly why review remains the developer's job.*